



JUNGLE JUMPER

Faith Dela Cruz



JUNGLE JUMPER

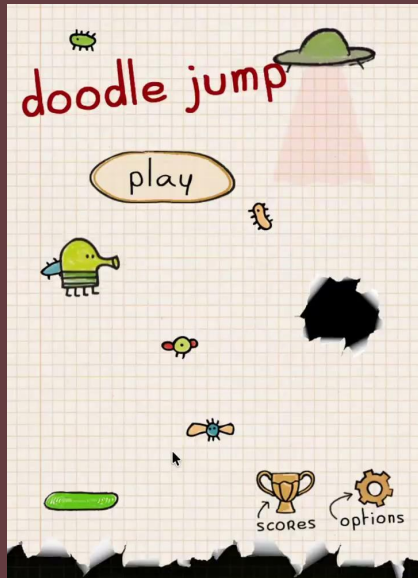
A 2D game platform with an infinite vertical scrolling.
Play as Monkey bouncing upward across platforms,
dodging obstacles, collecting power ups.





INSPIRE BY DOODLE JUMP

DOODLE JUMP



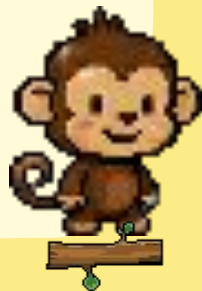
JUNGLE JUMPER





GAMEPLAY

1. Climb high by bouncing across platforms - normal, moving or broken.
2. Falling off the screen ends the run.
3. Watch out for birds, falling rocks, and bee swarms as score get higher
4. Collect bananas for ammo, springs for a boost, and shield for protection.
5. Try climb higher than your previous high score run.



DESIGN CONSIDERATIONS



DESIGN CONSIDERATIONS

Obstacle Timing

Interval for each obstacle and how they spawn

Ex: Bee Nests don't start release countdown until player is actually near

Score Difficulty

When each things appear so it's not overwhelming

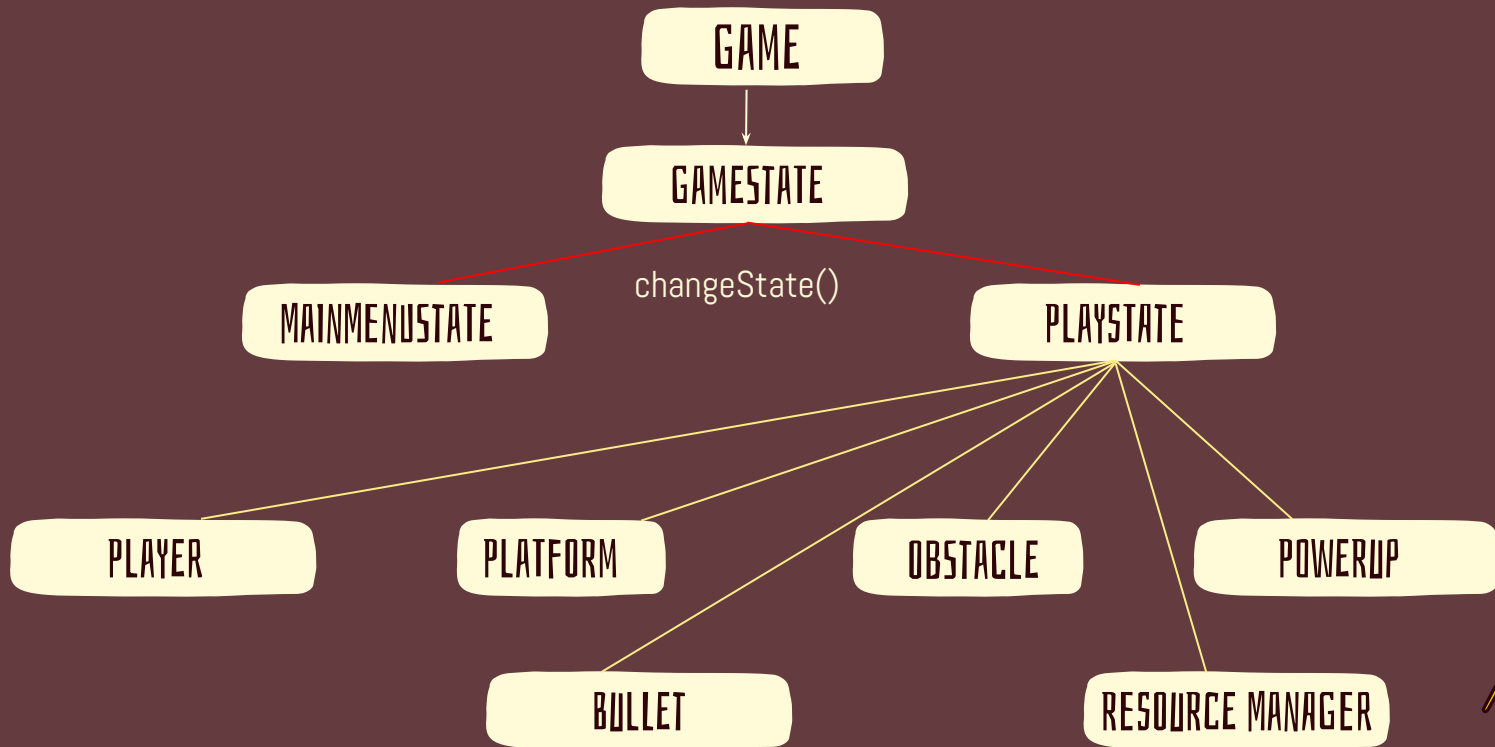
Obstacles and power ups unlock score threshold so difficulty ramp up gradually.

Background scrolling (still need to improve)

2 background identical background created, ground and repeating background.

CLASSES STRUCTURE

/ INHERITANCE (IS-A)
/ COMPOSITION (HAS-A)

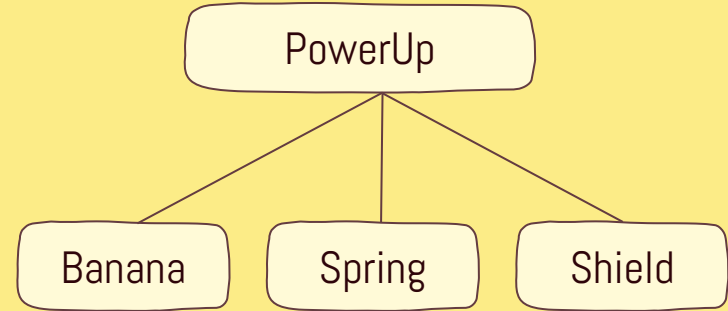
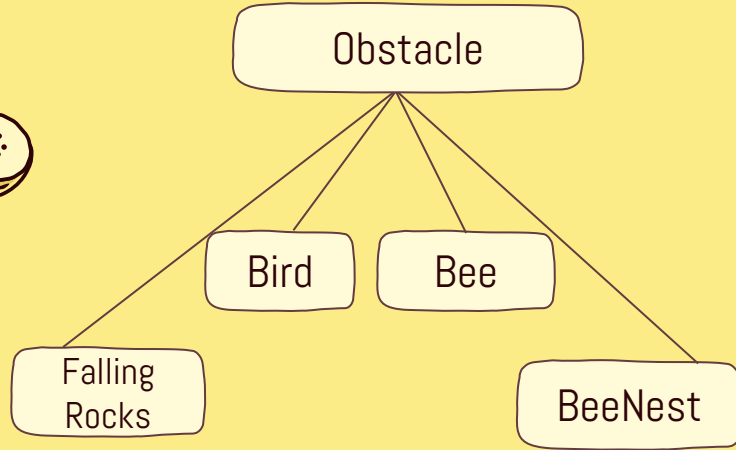


INHERITANCE COMPOSITIONS

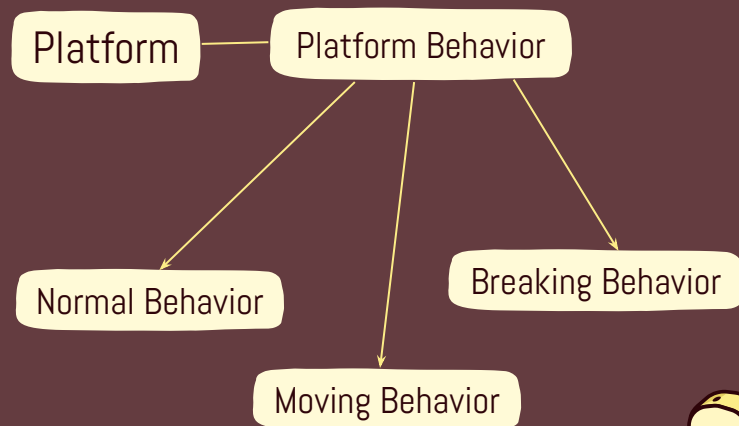
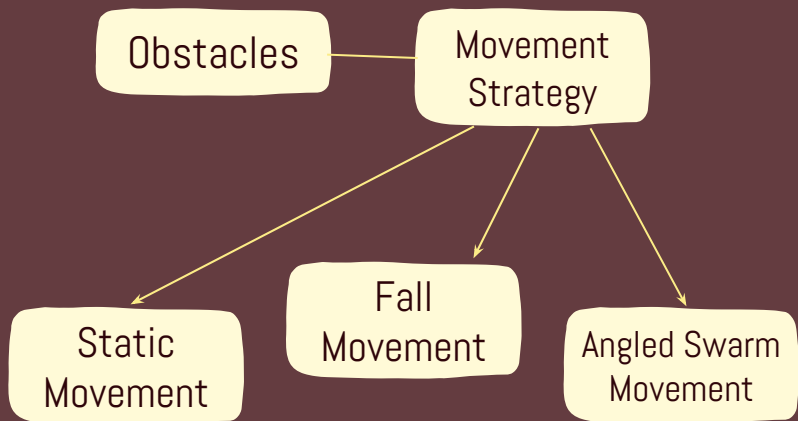




INHERITANCE



COMPOSITION/INHERITANCE



DESIGN PATTERNS



1. OBJECT POOL

```
11 class BulletPool {
12 public:
13     explicit BulletPool(int poolSize = 20);
14
15     //activate one currently-inactive bullet, if any are available, doesnt do anything if pool is currently in use
16     void fire(sf::Vector2f position);
17
18     //moves all active bullet and deactivate any that have scrolled above the top of visible camera view
19     void update(float deltaTime, float topOfViewY);
20
21     void draw(sf::RenderWindow& window) const;
22
23     //exposed so playstate can check bullet vs. obstacle collisions
24     std::vector<Bullet>& getBullets();
25
26 private:
27     std::vector<Bullet> bullets; //fixed size after construction
28 };
29
30 #endif //BULLET_POOL_HPP
31
```

- Without pooling: every shot creates new bullet then destroy it once it scroll off - constant allocation cause lag
- BulletPool pre-allocate set of 20 bullets once, None are every create/destroy again
- fire() scans for inactive bullet and reactivates it in place
- update() deactivate bullet as they scroll above camera return to pool

2. SINGLETON (RESOURCE MANAGER)

```
class ResourceManager {
public:
    //delete copy/move
    ResourceManager(const ResourceManager&) = delete;
    ResourceManager& operator=(const ResourceManager&) = delete;

    static ResourceManager& getInstance() {
        static ResourceManager instance; //created once on first use
        return instance;
    }

    sf::Texture& getTexture(const std::string& path) {
        auto it = textureCache.find(path);
        if (it != textureCache.end()) {
            return it->second; //
        }

        sf::Texture texture;
        if (!texture.loadFromFile(path)) {
            std::cerr << "ResourceManager: failed to load " << path << std::endl;
        }
        auto result = textureCache.emplace(path, std::move(texture));
        return result.first->second;
    }

private:
    ResourceManager() = default; //private constructor, only get instance() can create
    std::unordered_map<std::string, sf::Texture> textureCache;
};
```

- Platform, obstacles, - lots of object need same texture. Loading each one own copy waste load time and memory.
- Any class calls:
 `ResourceManager::getInstance(). getTexture(path)`
- One shared texture: first called loads and caches a texture, and every later caller - from anywhere, gets the same copy.

3. FACTORY



```
class ObstacleFactory {
public:
    static std::unique_ptr<Obstacle>
        createBird(sf::Vector2f position);
    static std::unique_ptr<Obstacle>
        createFallingRock(sf::Vector2f position, float fallSpeed);
    static std::unique_ptr<Obstacle>
        createBee(sf::Vector2f position, float angleRadians, float speed);
    static std::unique_ptr<Obstacle>
        createBeeNest(sf::Vector2f position);
};

#endif
```

- 4 Obstacle, each obstacle need different information to be built
- One class that building obstacles
- One static method per obstacle type: each type wraps a make_unique call internally.
- Every method returns a base

`unique_ptr<Obstacle>`

so PlayState drops whatever comeback into 1 shared list without caring the concrete type.

- Adding new obstacle later means touch one file, not every call site.





4. STRATEGY



```
class MovementStrategy {  
public:  
    virtual ~MovementStrategy() = default;  
    virtual void move (sf::RectangleShape& shape, float deltaTime) = 0;  
};
```

```
✓ Bird::Bird(sf::Vector2f position) : Obstacle(position, sf::Vector2f(10, 10),  
    std::make_unique<StaticMovement>(), "resources/bird.png") {  
}
```

```
void Obstacle::update(float deltaTime) {  
    movementStrategy->move(shape, deltaTime);  
    onUpdate(deltaTime);  
}
```

- Instead of separate movement code from each obstacle class
- Behavior pulled into separate, swappable object instead of subclass
- If two obstacle need to move the same, movement code gets reused not rewritten
- Obstacle::update() calls strategy->move() without knowing which concrete obstacle its moving

Used twice: Platform has its own Platform Behavior - same pattern but chosen randomly at spawn rather than fixed per type





FUTURE PLANS/IF GIVEN TIME



- Add music - attempted but audio issue
- Smooth out the background scrolling transition between two pieces
- Possibly add a twist to the core gameplay loop (SPRING)
- Handle score that go beyond the current expected/high score range (BIG BOSS)



RECORDED / LIVE DEMO



RECORDED:



THANK YOU!!
ANY QUESTIONS

